

Week .1

Course Name : Problem Solving and Testing

Course Code : 10211CS227

Name : SK NAZEEM

Vtu No : VTU28507

Session 1 :

1.Java Lambda Expressions.

Session 2 :

1.Java Sort.

2.Sort Array By Parity.

Session 3 :

1.Sorting:Comparator.

2.Java Comparator.

Session 4 :

1.Running Sum of ID Array.

2.Richest customer Wealth.

Session 5 :

1.Squares of a Sorted Array.

2.Find Pivot Index.

Session 1 :

1.Java Lambda Expressions.

PROGRAM :

```
import java.util.*;

interface PerformOperation {
    boolean check(int a);
}

class MyMath {

    public PerformOperation isOdd() {
        return n -> n % 2 != 0;
    }

    public PerformOperation isPrime() {
        return n -> {
            if (n < 2) return false;
            for (int i = 2; i * i <= n; i++) {
                if (n % i == 0) return false;
            }
            return true;
        };
    }

    public PerformOperation isPalindrome() {
        return n -> {
            int rev = 0, temp = n;
```

```

        while (temp > 0) {
            rev = rev * 10 + temp % 10;
            temp /= 10;
        }
        return rev == n;
    };
}

```

```

public boolean checker(PerformOperation p, int num) {
    return p.check(num);
}
}

```

```

public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        MyMath ob = new MyMath();

        int T = sc.nextInt();

        while (T-- > 0) {
            int ch = sc.nextInt();
            int num = sc.nextInt();

            if (ch == 1)
                System.out.println(ob.checker(ob.isOdd(), num) ? "ODD" : "EVEN");
            else if (ch == 2)
                System.out.println(ob.checker(ob.isPrime(), num) ? "PRIME" :
"COMPOSITE");

```

```
        else

            System.out.println(ob.checker(ob.isPalindrome(), num) ? "PALINDROME" :
"NOT PALINDROME");

        }

        sc.close();

    }

}
```

OUTPUT :

A screenshot of a Java IDE's output window. The window has a title bar with a dropdown menu, a search icon, a settings icon, and a close icon. The title is "input". The output text is as follows:

```
12
34
56
NOT PALINDROME
```

Session 2 :

1.Java Sort.

PROGRAM :

```
import java.util.*;
```

```
class Student {
```

```
    private int id;
```

```
    private String fname;
```

```
    private double cgpa;
```

```
    public Student(int id, String fname, double cgpa) {
```

```
        this.id = id;
```

```
        this.fname = fname;
```

```
        this.cgpa = cgpa;
```

```
    }
```

```
    public int getId() {
```

```
        return id;
```

```
    }
```

```
    public String getFname() {
```

```
        return fname;
```

```
    }
```

```
    public double getCgpa() {
```

```
        return cgpa;
```

```
    }
```

```
}
```

```

public class Solution {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());

        List<Student> studentList = new ArrayList<>();

        while (testCases-- > 0) {
            int id = in.nextInt();
            String fname = in.next();
            double cgpa = in.nextDouble();

            Student st = new Student(id, fname, cgpa);
            studentList.add(st);
        }

        Collections.sort(studentList, (s1, s2) -> {
            if (Double.compare(s2.getCgpa(), s1.getCgpa()) != 0) {
                return Double.compare(s2.getCgpa(), s1.getCgpa());
            }
            if (!s1.getFname().equals(s2.getFname())) {
                return s1.getFname().compareTo(s2.getFname());
            }
            return Integer.compare(s1.getId(), s2.getId());
        });

        for (Student s : studentList) {
            System.out.println(s.getFname());
        }
    }
}

```

```
}  
  
in.close();  
}  
}
```

OUTPUT :



The screenshot shows a code editor interface with two tabs: "Output" and "Generated files". The "Output" tab is active, displaying the following text on a black background:

```
1  
101 surya 2.5  
surya  
|
```

Below the output, a status bar indicates: "Compiled and executed in 11.719 sec(s)".

2.Sorted Array by Parity.

PROGRAM :

```
class Solution {
    public int[] sortArrayByParity(int[] nums) {
        int[] result = new int[nums.length];
        int index = 0;

        // Add even numbers first
        for (int num : nums) {
            if (num % 2 == 0) {
                result[index++] = num;
            }
        }

        // Add odd numbers
        for (int num : nums) {
            if (num % 2 != 0) {
                result[index++] = num;
            }
        }

        return result;
    }
}
```


OUTPUT:

 |  Problem List   

☒ Testcase |  Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[3,1,2,4]

Output

[2,4,3,1]

Expected

[2,4,3,1]

☒ Testcase |  **Test Result**

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

```
nums =  
[0]
```

Output

```
[0]
```

Expected

```
[0]
```

Session 3 :

1.Sorting : Comparator.

PROGRAM :

```
import java.util.*;
```

```
class Player {
```

```
    String name;
```

```
    int score;
```

```
    Player(String name, int score) {
```

```
        this.name = name;
```

```
        this.score = score;
```

```
    }
```

```
}
```

```
class Checker implements Comparator<Player> {
```

```
    @Override
```

```
    public int compare(Player a, Player b) {
```

```
        if (a.score != b.score) {
```

```
            return b.score - a.score; // Descending score
```

```
        }
```

```
        return a.name.compareTo(b.name); // Ascending name
```

```
    }
```

```
}
```

```
public class Solution {
```

```
    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);

int n = sc.nextInt();

Player[] player = new Player[n];

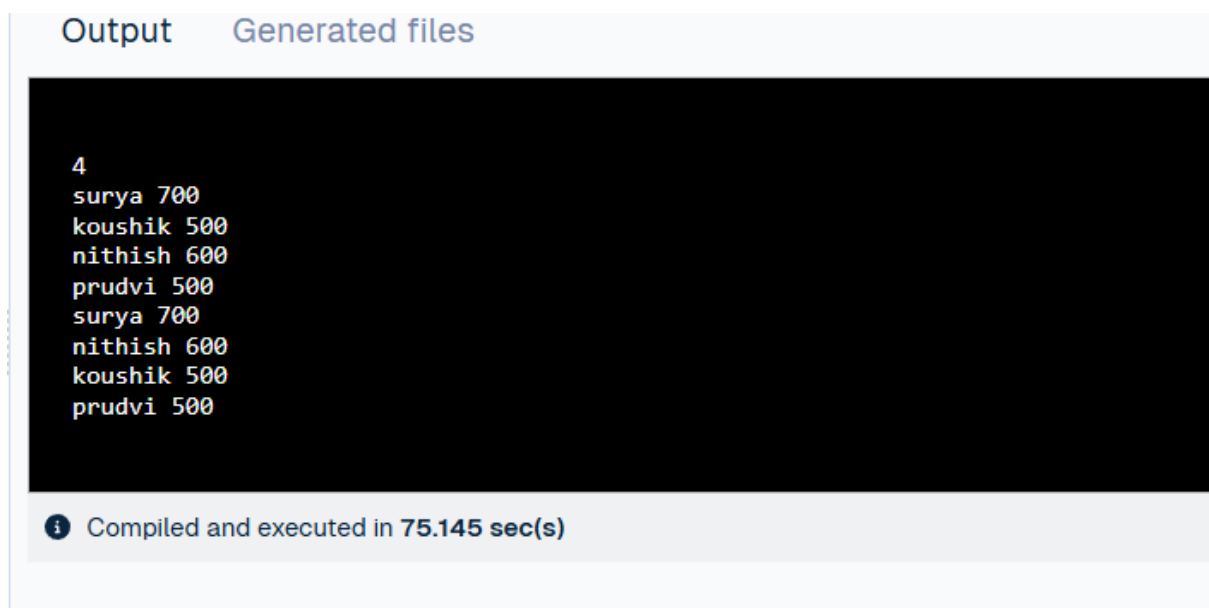
for (int i = 0; i < n; i++) {
    player[i] = new Player(sc.next(), sc.nextInt());
}

Arrays.sort(player, new Checker());

for (Player p : player) {
    System.out.println(p.name + " " + p.score);
}

sc.close();
}
```

OUTPUT :



```
Output    Generated files

4
surya 700
koushik 500
nithish 600
prudvi 500
surya 700
nithish 600
koushik 500
prudvi 500

i Compiled and executed in 75.145 sec(s)
```


2.Java Comparator.

PROGRAM :

```
import java.util.*;

class Player {
    String name;
    int score;

    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

class Checker implements Comparator<Player> {
    @Override
    public int compare(Player a, Player b) {
        if (a.score != b.score)
            return b.score - a.score;
        return a.name.compareTo(b.name);
    }
}

public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
```

```

Player[] player = new Player[n];

for (int i = 0; i < n; i++) {
    player[i] = new Player(sc.next(), sc.nextInt());
}

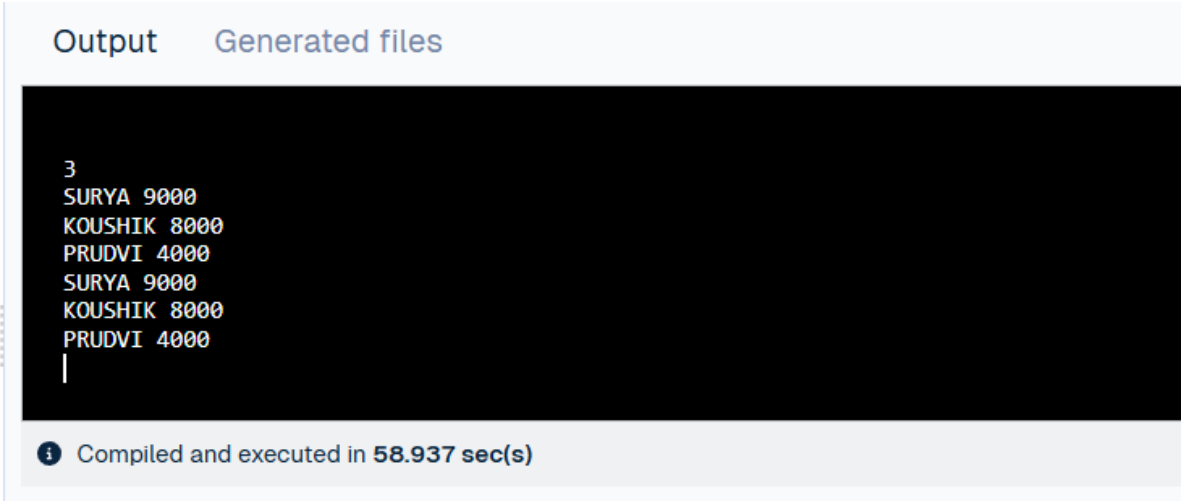
Arrays.sort(player, new Checker());

for (Player p : player) {
    System.out.println(p.name + " " + p.score);
}

sc.close();
}
}

```

OUTPUT :



The screenshot shows a code execution interface with two tabs: "Output" and "Generated files". The "Output" tab is active, displaying the following text:

```

3
SURYA 9000
KOUSHIK 8000
PRUDVI 4000
SURYA 9000
KOUSHIK 8000
PRUDVI 4000
|

```

Below the output, a status bar indicates: "Compiled and executed in 58.937 sec(s)".

Session 4:

1. Running Sum of ID Array.

PROGRAM :

```
class Solution {  
    public int[] runningSum(int[] nums) {  
        for (int i = 1; i < nums.length; i++) {  
            nums[i] = nums[i] + nums[i - 1];  
        }  
        return nums;  
    }  
}
```


OUTPUT :

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

nums =
[1,2,3,4]

Output

[1,3,6,10]

Expected

[1,3,6,10]

☒ Testcase |  **Test Result**

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
nums =  
[1,1,1,1,1]
```

Output

```
[1,2,3,4,5]
```

Expected

```
[1,2,3,4,5]
```

☒ Testcase | **>_ Test Result**

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
nums =  
[3,1,2,10,1]
```

Output

```
[3,4,6,16,17]
```

Expected

```
[3,4,6,16,17]
```

2.Richest Customer Wealth.

PROGRAM :

```
class Solution {  
    public int maximumWealth(int[][] accounts) {  
        int maxWealth = 0;  
  
        for (int i = 0; i < accounts.length; i++) {  
            int sum = 0;  
            for (int j = 0; j < accounts[i].length; j++) {  
                sum += accounts[i][j];  
            }  
            if (sum > maxWealth) {  
                maxWealth = sum;  
            }  
        }  
  
        return maxWealth;  
    }  
}
```

OUTPUT :

☒ Testcase |  **Test Result**

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
accounts =  
[[1,2,3],[3,2,1]]
```

Output

6

Expected

6

☒ Testcase |  **Test Result**

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
accounts =  
[[1,5],[7,3],[3,5]]
```

Output

```
10
```

Expected

```
10
```

 |  Problem List   

☒ Testcase |  Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
accounts =  
[[2,8,7],[7,1,3],[1,9,5]]
```

Output

```
17
```

Expected

```
17
```

Session 5 :

1.Squares of a Sorted Array.

PROGRAM :

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        int left = 0;
        int right = n - 1;
        int index = n - 1;

        while (left <= right) {
            if (Math.abs(nums[left]) > Math.abs(nums[right])) {
                result[index] = nums[left] * nums[left];
                left++;
            } else {
                result[index] = nums[right] * nums[right];
                right--;
            }
            index--;
        }

        return result;
    }
}
```

OUTPUT :

☒ Testcase  Test Result

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

```
nums =  
[-4,-1,0,3,10]
```

Output

```
[0,1,9,16,100]
```

Expected

```
[0,1,9,16,100]
```

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

```
nums =  
[-7,-3,2,3,11]
```

Output

```
[4,9,9,49,121]
```

Expected

```
[4,9,9,49,121]
```

2.Find Pivot Index.

PROGRAM :

```
class Solution {  
    public int pivotIndex(int[] nums) {  
        int total = 0;  
  
        // Calculate total sum  
        for (int num : nums) {  
            total += num;  
        }  
  
        int leftSum = 0;  
  
        // Find pivot index  
        for (int i = 0; i < nums.length; i++) {  
            if (leftSum == total - leftSum - nums[i]) {  
                return i;  
            }  
            leftSum += nums[i];  
        }  
  
        return -1;  
    }  
}
```

OUTPUT :

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
nums =  
[1,7,3,6,5,6]
```

Output

3

Expected

3

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
nums =  
[1,2,3]
```

Output

```
-1
```

Expected

```
-1
```

☒ Testcase [> Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
nums =  
[2,1,-1]
```

Output

```
0
```

Expected

```
0
```